Sun Search Point

Component Design Document

1 Description

Safe-mode sun search and pointing guidance. Runs a scripted sun-search rotation sequence, then transitions to closed-loop sun pointing once the sun is acquired or the sequence elapses.

2 Requirements

No requirements have been specified for this component.

3 Design

3.1 At a Glance

Below is a list of useful parameters and statistics that give a quick look into the makeup of the component.

- **Execution** - *passive*
- **Number of Connectors** - 5
- **Number of Invokee Connectors** - 3
- **Number of Invoker Connectors** - 2
- **Number of Generic Connectors** - *None*
- **Number of Generic Types** - *None*
- **Number of Unconstrained Arrayed Connectors** - *None*
- **Number of Commands** - *None*
- **Number of Parameters** - 6
- **Number of Events** - *None*
- **Number of Faults** - *None*
- **Number of Data Products** - 2
- **Number of Data Dependencies** - 3
- **Number of Packets** - *None*

3.2 Diagram



Figure 1: Sun Search Point component diagram.

3.3 Connectors

Below are tables listing the component's connectors.

3.3.1 Invokee Connectors

The following is a list of the component's *invokee* connectors:

Table 1: Sun Search Point Invokee Connectors

Name	Kind	Type	Return_Type	Count
Tick_T_Recv_Sync	recv_sync	Tick.T	-	1
Parameter_ Update_T_Modify	modify	Parameter_ Update.T	-	1
Reset_Tick_T_ Recv_Sync	recv_sync	Tick.T	-	1

Connector Descriptions:

- **Tick_T_Recv_Sync** - Run the algorithm up to the current time.
- **Parameter_Update_T_Modify** - The parameter update connector.
- **Reset_Tick_T_Recv_Sync** - Re-arm the sun search sequence so the next tick restarts at the first rotation. Must be called on any transition, or re-entrance, to safe state.

3.3.2 Invoker Connectors

The following is a list of the component's *invoker* connectors:

Table 2: Sun Search Point Invoker Connectors

Name	Kind	Type	Return_Type	Count
Data_Product_ Fetch_T_Request	request	Data_Product_ Fetch.T	Data_Product_ Return.T	1
Data_Product_T_ Send	send	Data_Product.T	-	1

Connector Descriptions:

- **Data_Product_Fetch_T_Request** - Fetch a data product item from the database.
- **Data_Product_T_Send** - The data product invoker connector

3.4 Interrupts

This component contains no interrupts.

3.5 Initialization

Below are details on how the component should be initialized in an assembly.

3.5.1 Component Instantiation

This component contains no instantiation parameters in its discriminant.

3.5.2 Component Base Initialization

This component contains no base class initialization, meaning there is no `init_Base` subprogram for this component.

3.5.3 Component Set ID Bases

This component contains commands, events, packets, faults, or data products that require a base identifier to be set at initialization. The `set_Id_Bases` procedure must be called with the following parameters:

Table 3: Sun Search Point Set Id Bases Parameters

Name	Type
`Data_Product_Id_Base`	`Data_Product_Types.Data_Product_Id_Base`
`Parameter_Id_Base`	`Parameter_Types.Parameter_Id_Base`

Parameter Descriptions:

- **`Data_Product_Id_Base`** - The value at which the component's data product identifiers begin.
- **`Parameter_Id_Base`** - The value at which the component's parameter identifiers begin.

3.5.4 Component Map Data Dependencies

This component contains data dependencies that need their IDs mapped to valid data product IDs at initialization. Each data dependency also needs a stale limit provided in microseconds. If the timestamp of the data dependency is older than the reference time (usually the current time) minus the stale limit then the data dependency is considered too old to safely use. The `map_Data_Dependencies` procedure must be called with the following parameters:

Table 4: Sun Search Point Map Data Dependencies Parameters

Name	Type
`Omega_Bn_B_Id`	`Data_Product_Types.Data_Product_Id`
`Omega_Bn_B_Stale_Limit`	`Ada.Real_Time.Time_Span`
`R_Hat_Sb_B_Id`	`Data_Product_Types.Data_Product_Id`
`R_Hat_Sb_B_Stale_Limit`	`Ada.Real_Time.Time_Span`
`Num_Css_Viewing_Sun_Id`	`Data_Product_Types.Data_Product_Id`
`Num_Css_Viewing_Sun_Stale_Limit`	`Ada.Real_Time.Time_Span`

Parameter Descriptions:

- **`Omega_Bn_B_Id`** - The external data product ID used to fetch the value for the 'Omega_Bn_B' data dependency.
- **`Omega_Bn_B_Stale_Limit`** - The stale limit used for the 'Omega_Bn_B' data dependency. If the fetched data product is older than the given Time_Span then it is deemed stale and unfit for use by the component.
- **`R_Hat_Sb_B_Id`** - The external data product ID used to fetch the value for the 'R_Hat_Sb_B' data dependency.

- **R_Hat_Sb_B_Stale_Limit** - The stale limit used for the 'R_Hat_Sb_B' data dependency. If the fetched data product is older than the given Time_Span then it is deemed stale and unfit for use by the component.
- **Num_Css_Viewing_Sun_Id** - The external data product ID used to fetch the value for the 'Num_Css_Viewing_Sun' data dependency.
- **Num_Css_Viewing_Sun_Stale_Limit** - The stale limit used for the 'Num_Css_Viewing_Sun' data dependency. If the fetched data product is older than the given Time_Span then it is deemed stale and unfit for use by the component.

3.5.5 Component Implementation Initialization

The calling of this implementation class initialization procedure is mandatory. Initializes the sun search point algorithm with the control period and the default parameter values. The `init` subprogram requires the following parameters:

Table 5: Sun Search Point Implementation Initialization Parameters

Name	Type	Default Value
`Control_Period`	`Basic_Types.Positive_Short_Float`	*None provided*

Parameter Descriptions:
- **Control_Period** - [s] Time between two algorithm updates. Fixed by the rate at which the assembly schedules the component, so it is not a runtime parameter. The type keeps it greater than zero.

3.6 Commands

The Sun Search Point component has no commands.

3.7 Parameters

Parameters for the Sun Search Point component

Table 6: Sun Search Point Parameters

Local ID	Parameter Name	Type	Default Value
0x0000 (0)	Rotations	Rotation_Properties_X4_Record.T	(Rotations=>[others=>(Rotation_Duration=>1.0, Rotation_Rate=>0.0,Rotation_Axis=>Sun_Search_Point_Enums.Body_Axis.B1_Hat_B)])
0x0001 (1)	S_Hat_Bdy_Cmd	Packed_F32x3.T	[0.0,0.0,1.0]
0x0002 (2)	Sun_Axis_Spin_Rate	Packed_F32.T	(Value=>0.0)
0x0003 (3)	Omega_Rn_B_Cfg	Packed_F32x3.T	[0.0,0.0,0.0]

0x0004 (4)	Observation_Threshold	Packed_ Observation_ Threshold.T	(Value=>4)
0x0005 (5)	Deadband_Angle	Packed_F32.T	(Value=>0.0)

Parameter Descriptions:

- **Rotations** - [-] The sun-search rotation sequence, in the order commanded. The search runs each entry for its duration, commanding that rate about that body axis. Each duration must be finite and greater than zero, and each rate must be finite.
- **S_Hat_Bdy_Cmd** - [-] The body-fixed unit vector that is driven onto the measured sun direction during the pointing phase. Norm must be within 1e-3 of 1.0.
- **Sun_Axis_Spin_Rate** - [rad/s] Signed spin rate commanded about the measured sun direction once the sun is acquired. The commanded reference rate in the pointing phase is this scalar times the sun unit vector, so zero holds the vehicle inertially fixed on the sun line. Must be finite.
- **Omega_Rn_B_Cfg** - [rad/s] Body rate commanded during the pointing phase when the sun direction is unavailable. The sun direction input is the exact zero vector when no coarse sun sensor sees the sun, and this rate is commanded instead. Must be finite.
- **Observation_Threshold** - [-] Number of coarse sun sensors that must see the sun, at or above which the sun is considered acquired and the search hands over to pointing. Checked only after the first rotation completes. Zero hands over as soon as the first rotation completes.
- **Deadband_Angle** - [rad] Attitude deadband applied during the pointing phase. The commanded attitude error is the sun angle error less this band, and zero inside it, so the controller holds the vehicle at the band edge rather than driving the error to zero. Must be finite and not negative.

3.8 Events

The Sun Search Point component has no events.

3.9 Data Products

Data products for the Sun Search Point component.

Table 7: Sun Search Point Data Products

Local ID	Data Product Name	Type
0x0000 (0)	Attitude_Guidance	Att_Guid.T
0x0001 (1)	Sun_Search_Status	Packed_Sun_Search_Status.T

Data Product Descriptions:

- **Attitude_Guidance** - The attitude guidance for the attitude controller: the attitude error (MRPs) of the body frame relative to the reference frame, the body rate error of B relative to R, and the reference frame rate of R relative to N, all in B frame components. The reference frame acceleration is zero, since the search and pointing rates are piecewise constant. Published every tick.
- **Sun_Search_Status** - Whether the search sequence acquired the sun. Published every tick, and latches Sun_Not_Found once the sequence elapses without an acquisition until the reset connector re-arms the search.

3.10 Data Dependencies

Data dependencies for the Sun Search Point component.

Table 8: Sun Search Point Data Dependencies

Data Dependency Name	Type
Omega_Bn_B	Packed_F32x3.T
R_Hat_Sb_B	Packed_F32x3.T
Num_Css_Viewing_Sun	Packed_U32.T

Data Dependency Descriptions:

- **Omega_Bn_B** - Measured body rate.
- **R_Hat_Sb_B** - Sun direction vector in the body frame.
- **Num_Css_Viewing_Sun** - Number of coarse sun sensors observing the sun this cycle. Drives the transition out of the search phase.

3.11 Packets

The Sun Search Point component has no packets.

3.12 Faults

The Sun Search Point component has no faults.

4 Unit Tests

The following section describes the unit test suites written to test the component.

4.1 *Sun_Search_Point_Tests* Test Suite

This is a unit test suite for the Sun Search Point component

Test Descriptions:

- **Test** - Run the search phase to ensure integration is sound.
- **Test_Search_To_Point_Transition** - Verify the search phase transitions to pointing once the observation count reaches the threshold after the first rotation.
- **Test_Forced_Transition_Sets_Fault** - Verify the full rotation sequence elapsing without acquiring the sun forces pointing and latches the search-failure flag.
- **Test_Reset** - Verify the reset connector re-arms the search sequence after the terminal pointing phase.
- **Test_Deadband** - Verify the pointing phase controls to the edge of the attitude deadband, commanding no attitude error inside the band and the error less the band outside it.
- **Test_Invalid_Parameter** - Ensure staged parameter values the algorithm would reject are refused at validation.
- **Test_Observation_Threshold_Range** - Verify an observation threshold above the coarse sun sensor count is refused.
- **Test_Invalid_Data_Dependency** - Ensure a data dependency with the wrong identifier is treated as a wiring defect and fails the tick's assertion.

5 Appendix

5.1 Preamble

This component contains no preamble code.

5.2 Packed Types

The following section outlines any complex data types used in the component in alphabetical order. This includes packed records and packed arrays that might be used as connector types, command arguments, event parameters, etc..

Att_Guid.T:

Table 9: Att_Guid Packed Record : 384 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Sigma_Br	Packed_F32x3.T	-	96	0	95
Omega_Br_B	Packed_F32x3.T	-	96	96	191
Omega_Rn_B	Packed_F32x3.T	-	96	192	287
Domega_Rn_B	Packed_F32x3.T	-	96	288	383

Field Descriptions:

- **Sigma_Br** - *No description provided.*
- **Omega_Br_B** - *No description provided.*
- **Omega_Rn_B** - *No description provided.*
- **Domega_Rn_B** - *No description provided.*

Data_Product.T:

Generic data product packet for holding arbitrary data types

Table 10: Data_Product Packed Record : 1096 bits *(maximum)*

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Data_Product_ Header.T	-	72	0	71	-
Buffer	Data_Product_ Types.Data_ Product_ Buffer_Type	-	1024	72	1095	Header. Buffer_Length

Field Descriptions:

- **Header** - The data product header
- **Buffer** - A buffer that contains the data product type

Data_Product_Fetch.T:

A packed record which holds information for a data product request.

Table 11: Data_Product_Fetch Packed Record : 16 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Id	Data_Product_Types. Data_Product_Id	0 to 65535	16	0	15

Field Descriptions:

- **Id** - The data product identifier

Data_Product_Header.T:

Generic data_product packet for holding arbitrary data_product types

Table 12: Data_Product_Header Packed Record : 72 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	48	0	47
Id	Data_Product_Types. Data_Product_Id	0 to 65535	16	48	63
Buffer_Length	Data_Product_ Types.Data_Product_ Buffer_Length_Type	0 to 128	8	64	71

Field Descriptions:

- **Time** - The timestamp for the data product item.
- **Id** - The data product identifier
- **Buffer_Length** - The number of bytes used in the data product buffer

Data_Product_Return.T:

This record holds data returned from a data product fetch request.

Table 13: Data_Product_Return Packed Record : 1104 bits *(maximum)*

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
The_ Status	Data_ Product_ Enums. Fetch_ Status.E	0 => Success 1 => Not_Available 2 => Id_Out_Of_Range	8	0	7	-
The_ Data_ Product	Data_ Product. T	-	1096	8	1103	-

Field Descriptions:

- **The_Status** - A status relating whether or not the data product fetch was successful or not.
- **The_Data_Product** - The data product item returned.

Packed_F32.T:

Single component record for holding packed 32-bit floating point number.

Table 14: Packed_F32 Packed Record : 32 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Value	Short_Float	-3.40282e+38 to 3.40282e+38	32	0	31

Field Descriptions:

- **Value** - The 32-bit floating point number.

Packed_F32x3.T:

Packed 3 element vector of 32-bit floats.

Table 15: Packed_F32x3 Packed Array : 96 bits

Type	Range	Element Size (Bits)	Length	Total Size (Bits)
Short_Float	-3.40282e+38 to 3.40282e+38	32	3	96

Packed_Observation_Threshold.T:

Coarse sun sensor observation count threshold, bounded by the number of sensors in the system. Values outside the range fail type validation during parameter staging. *Preamble (inline Ada definitions):*

```
-- The threshold counts coarse sun sensors observing the sun, so it can never
 ↪  usefully exceed the
-- number of sensors in the system. The upper bound mirrors MAX_NUM_CSS_SENSORS
 ↪  on the C side, and
-- Sun_Search_Point_Algorithm_C asserts at elaboration that the two still agree.
subtype Observation_Threshold_Type is Natural range 0 .. 8;
```

Table 16: Packed_Observation_Threshold Packed Record : 8 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Value	Observation_ Threshold_Type	0 to 8	8	0	7

Field Descriptions:

- **Value** - The observation count threshold.

Packed_Sun_Search_Status.T:

Sun search and pointing status, packed as an 8-bit enumeration.

Table 17: Packed_Sun_Search_Status Packed Record : 8 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit

Value	Sun_Search_ Point_Enums. Sun_Search_ Status.E	0 => Sun_Found 1 => Sun_Not_Found	8	0	7

Field Descriptions:
- **Value** - The sun search and pointing status.

Packed_U32.T:

Single component record for holding packed unsigned 32-bit value.

Table 18: Packed_U32 Packed Record : 32 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Value	Interfaces. Unsigned_32	0 to 4294967295	32	0	31

Field Descriptions:
- **Value** - The 32-bit unsigned integer.

Parameter.T:

Generic parameter packet for holding a generic parameter

Table 19: Parameter Packed Record : 792 bits *(maximum)*

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Parameter_ Header.T	-	24	0	23	-
Buffer	Parameter_ Types. Parameter_ Buffer_Type	-	768	24	791	Header.Buffer_ Length

Field Descriptions:
- **Header** - The parameter header
- **Buffer** - A buffer that contains the parameter type

Parameter_Header.T:

Generic parameter header for holding arbitrary parameters

Table 20: Parameter_Header Packed Record : 24 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Id	Parameter_Types. Parameter_Id	0 to 65535	16	0	15

Buffer_Length	Parameter_Types. Parameter_Buffer_ Length_Type	0 to 96	8	16	23

Field Descriptions:

- **Id** - The parameter identifier
- **Buffer_Length** - The number of bytes used in the parameter type buffer

Parameter_Update.T:

A record intended to be used as a provide/modify connector type for updating/fetching parameters.

Table 21: Parameter_Update Packed Record : 824 bits *(maximum)*

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Table_Id	Parameter_ Types. Parameter_ Table_Id	0 to 65535	16	0	15	-
Operation	Parameter_ Enums. Parameter_ Operation_ Type.E	0 => Stage 1 => Update 2 => Fetch 3 => Validate	8	16	23	-
Status	Parameter_ Enums. Parameter_ Update_ Status.E	0 => Success 1 => Id_Error 2 => Validation_Error 3 => Length_Error	8	24	31	-
Param	Parameter. T	-	792	32	823	-

Field Descriptions:

- **Table_Id** - The ID for the table that contains this parameter
- **Operation** - The parameter operation to perform.
- **Status** - The parameter return status.
- **Param** - The parameter that has been updated or fetched.

Rotation_Properties.T:

Properties of a single sun-search rotation maneuver. Maps to RotationProperties_c in the sunSearchPoint C shim.

Table 22: Rotation_Properties Packed Record : 72 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit

Rotation_ Duration	Basic_ Types. Positive_ Short_Float	2.58494e-26 to 1.93428e+25	32	0	31
Rotation_ Rate	Short_Float	-3.40282e+38 to 3.40282e+38	32	32	63
Rotation_ Axis	Sun_Search_ Point_ Enums.Body_ Axis.E	0 => B1_Hat_B 1 => B2_Hat_B 2 => B3_Hat_B	8	64	71

Field Descriptions:

- **Rotation_Duration** - [s] Duration of this rotation. Must be finite and greater than zero.
- **Rotation_Rate** - [rad/s] Signed scalar body rate commanded during this rotation. Must be finite.
- **Rotation_Axis** - [-] Body axis about which to rotate.

Rotation_Properties_X4.T:

The four sun-search rotations, in the order they are commanded.

Table 23: Rotation_Properties_X4 Packed Array : 288 bits

Type	Range	Element Size (Bits)	Length	Total Size (Bits)
Rotation_Properties.T	-	72	4	288

Rotation_Properties_X4_Record.T:

Record wrapping the sun-search rotation sequence. Maps to RotationPropertiesArray4_c in the sunSearchPoint C shim, which passes it by pointer.

Table 24: Rotation_Properties_X4_Record Packed Record : 288 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Rotations	Rotation_Properties_ X4.T	-	288	0	287

Field Descriptions:

- **Rotations** - The sun-search rotation sequence.

Sys_Time.T:

A record which holds a time stamp using EMA VTC format including seconds and subseconds since epoch (1-5-1980 to 1-6-1980 midnight).

Table 25: Sys_Time Packed Record : 48 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit

Seconds	Interfaces. Unsigned_32	0 to 4294967295	32	0	31
Subseconds	Interfaces. Unsigned_16	0 to 65535	16	32	47

Field Descriptions:

- **Seconds** - The number of seconds elapsed since epoch.
- **Subseconds** - The number of 1/(2^16) sub-seconds.

Tick.T:

The tick datatype used for periodic scheduling. Included in this type is the Time associated with a tick and a count.

Table 26: Tick Packed Record : 80 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	48	0	47
Count	Interfaces. Unsigned_32	0 to 4294967295	32	48	79

Field Descriptions:

- **Time** - The timestamp associated with the tick.
- **Count** - The cycle number of the tick.

5.3 Enumerations

The following section outlines any enumerations used in the component.

Data_Product_Enums.Fetch_Status.E:

This status denotes whether a data product fetch was successful.

Table 27: Fetch_Status Literals:

Name	Value	Description
Success	0	The data product was returned successfully.
Not_Available	1	No data product is yet available for the provided id.
Id_Out_Of_Range	2	The data product id was out of range.

Parameter_Enums.Parameter_Operation_Type.E:

This enumeration lists the different parameter operations that can be performed.

Table 28: Parameter_Operation_Type Literals:

Name	Value	Description
Stage	0	Stage the parameter.
Update	1	All parameters are staged, it is ok to update all parameters now.

Fetch	2	Fetch the parameter.
Validate	3	Validate all the parameters.

Parameter_Enums.Parameter_Update_Status.E:

This status enumeration provides information on the success/failure of a parameter operation.

Table 29: Parameter_Update_Status Literals:

Name	Value	Description
Success	0	Parameter was successfully staged.
Id_Error	1	Parameter id was not valid.
Validation_Error	2	Parameter values were not successfully validated.
Length_Error	3	Parameter length was not correct.

Sun_Search_Point_Enums.Body_Axis.E:

Body axis about which a sun-search rotation is commanded. Values match the C RotationAxis_c enumeration.

Table 30: Body_Axis Literals:

Name	Value	Description
B1_Hat_B	0	Rotate about the first body axis.
B2_Hat_B	1	Rotate about the second body axis.
B3_Hat_B	2	Rotate about the third body axis.

Sun_Search_Point_Enums.Sun_Search_Status.E:

Outcome of the sun search sequence, latched once the sequence ends and cleared by the reset connector.

Table 31: Sun_Search_Status Literals:

Name	Value	Description
Sun_Found	0	The sun was acquired, or the search is still running.
Sun_Not_Found	1	The search sequence elapsed without acquiring the sun and pointing was forced.